Test Specification – Part A: Test Plan

Document purpose and current status

This Test Plan describes the strategy, scope, tools, environment, responsibilities and schedule used to verify the ABC Fashion AI Shopping Agent MVP against its Software Requirements Specification (SRS). Individual test inputs, expected outcomes and execution records are maintained separately in the Test Case Specifications document. Functionality and performance based testing has been performed on our prototype as available through the team GitHub repository (LINK GITHUB HERE). The test evidence is organised by layer so that automated regression, interface behaviour and service-level validation are assessed against their respective requirements.

1. Testing Strategy

Testing follows a requirements-based and risk-based approach. Test cases are mapped to the SRS use cases (UC01–UC07) and to relevant product behaviours. Priority is given to grounded product information, preservation of explicit customer constraints, size and budget enforcement, catalogue retrieval, state transitions in multi-turn shopping, and integration failures across the frontend, backend/API, Supabase and the OpenAI service.

Testing is performed in layers: developer-level unit/component tests; offline regression tests; API and integration checks; recommendation and conversation scenario testing; manual browser/UI and responsive checks; and client acceptance testing. These layers are reported separately because a passing component test does not prove a complete browser journey or production integration.

The repository regression runner is `npm test` (`node scripts/run-regression.mjs`). It runs 41 named offline test scripts using Node.js and `tsx`. As evidenced through testing, the recorded regression log reports that all 41 scripts passed.

The Test Case Specifications document records the functional cases with passing results in the supplied test record, alongside the repository's recorded automated regression evidence. Test artefacts should be retained in the project repository so the team can trace each result to its input and supporting evidence.

2. Test Types

The following test types cover the originally specified MVP flows and the expanded conversation and outfit-shopping behaviours present in the repository. Each type targets a distinct layer of product quality and is mapped to the relevant requirements.

Test type	Purpose	Scope and evidence
Functional testing	Verify user-facing features and use cases behave as specified.	Catalogue, Product Details, cart, AI Stylist, search/filter controls, product cards, selected outfit review and shopping actions. Manual cases remain separately tracked.
Unit/component testing	Verify deterministic functions and domain logic.	Preference validation, recommendation contracts, hard constraints, price/size validation, task state, conversation recovery, product references, presentation and cross-sell/upsell logic.
Regression testing	Detect regressions after code changes.	41 offline scripts defined by `scripts/run-regression.mjs`. Recorded successful run: 41/41 scripts passed.
Integration/API testing	Verify services and components exchange valid data.	Supabase catalogue retrieval, extraction/recommendation APIs, response validation, HTTP errors, OpenAI integration and live stock. Current offline runner explicitly excludes live/API checks.
Recommendation and conversation quality	Ensure customer requirements remain intact across interpretation, retrieval, ranking and follow-up turns.	Explicit hard constraints, budget, size, exclusions, category/colour/occasion/style, clarifications, collection-first onboarding, refinements, alternatives, no-match cases, grouped outfits and combined budgets.
UI, usability and compatibility	Verify visible behaviour and responsive layout.	Landing page, AI Stylist chat, search filters, product cards, Product Details and cart on desktop/mobile. Screenshots in repository are supporting evidence, not a substitute for a complete acceptance run.

Security and failure handling	Ensure invalid input and unavailable dependencies fail safely.	Malformed requests, unsupported content types, invalid limits, empty input, OpenAI/Supabase outages, credential exposure and controlled errors. Failure-handling scenarios are included in the overall quality strategy.
Acceptance testing	Confirm the delivered MVP meets client expectations.	End-to-end deployed journeys and final client demonstration/handover. Complements the automated regression suite with end-to-end confirmation against the agreed acceptance criteria.

3. Testing Tools

Tools are selected to provide repeatable automated evidence and realistic user-level checks across the development, integration and acceptance layers.

Tool / artifact	**Use**
Node.js and npm	Run the repository regression suite and project scripts.
`npm test` / `scripts/run-regression.mjs`	Execute 41 offline regression scripts and save a timestamped log under `docs/test-evidence/`.
TypeScript / ESLint / Next.js build	Check type correctness, lint quality and production build readiness; run separately and retain logs.
Supabase dashboard and catalogue	Inspect catalogue rows, sizes, availability and product specifications. Live connectivity should be recorded separately from fixture-based tests.
OpenAI API / extraction and conversation scripts	Test natural-language extraction and multi-turn response behaviour where credentials and service access are available. Live-model runs are distinct from deterministic offline tests.
Desktop/mobile browsers and developer tools	Manually test UI layout, links, cart updates, network responses and console errors. Browser journeys are not covered by the offline runner.

Vercel deployment	Test the deployed prototype and production-like flows after a deployment smoke test.
GitHub pull requests and Jira	Review changes, track defects, record owners and verify fixes.
Test Case Specifications and `docs/test-evidence/`	Record inputs, expected outcomes, actual observations, status, run date, and references to logs/screenshots/transcripts.

4. Test Environment and Resources

Deployment environment: the prototype URL recorded in the original plan is https://abcandco.vercel.app/. Verify availability at the time of testing; a URL alone is not evidence that the current deployment passes.

Development environment: Next.js/React application, Node.js and npm. The recorded successful regression run used Node v24.14.0 on macOS.

Test data: synthetic ABC Fashion catalogue and fixtures. Repository evidence records 316 publicly visible Supabase products in prior catalogue checks; this value should be reverified when running against the live catalogue.

External services: Supabase for catalogue data and OpenAI for natural-language interpretation. Credentials and network access are required for live integration tests; do not include secrets in evidence.

Hardware: desktop/laptop and mobile device or browser emulation for responsive checks.

Human resources: Sarah is identified in the earlier testing context as responsible for running the tests; retain the actual executor and date for each new execution. Automated repository tests should identify the script/harness as the executor and the person supervising the run separately.

5. Responsibilities

Responsibilities below preserve the roles in the supplied plan. When tests are rerun, the execution record should name the person who actually ran the test; automated logs should be attached rather than attributing a run to a team member without evidence.

Role	Team member(s)	Testing responsibility

Frontend developers	Salvi, Hans, Taylor	Functional, UI and compatibility testing of developed components.
AI & Recommendation Leads	Beatrice, Abrar	Preference extraction, recommendation quality, hard-constraint enforcement and AI integration testing.
Data & Testing Leads	Sarah, Taylor, Abrar	Test data preparation, test planning and overall system testing.
QA & Rubric Compliance	Sarah	Compile test documentation, ensure consistency with the SRS, and coordinate evidence capture.
Client	Peyton Bernhardt (Salesforce)	Acceptance testing during final demonstration or handover.
Repository regression runner	Automated Node.js scripts; supervised by the assigned developer/tester	Run deterministic offline checks and retain timestamped logs. Do not represent the run as live browser or API acceptance.

6. Milestones and Schedule

The schedule aligns testing activities with the project delivery milestones. Dates can be maintained by the project team as sprint and handover planning is finalised.

Milestone	**Testing activities**	**Target date / status**
Sprint 3	Component-level functional testing and initial integration testing.	[CONFIRM]
Deliverable 3	Test Plan and initial Test Case Specifications submitted.	1 October 2026 (historical milestone)

Pre-D4	Recommendation quality, usability, compatibility and regression testing; re-run after clean dependency installation.	[CONFIRM]
Deliverable 4 / Handover	Client acceptance testing and final test results.	[CONFIRM]

7. Recorded Execution Summary

Evidence / run	Result	Summary
Recorded regression log: `regression-2026-10-19T05-36-45-919Z.txt`	41/41 scripts passed.	Successful recorded offline regression run dated 19 October 2026.
Original functional test record	Passing cases retained in the Test Case Specifications document.	Functional results are presented with their case identifiers, expected behaviour and recorded observations.

The recorded offline regression result run by Group 20 on 19 October 2026 defines a robust and repeatable baseline baseline for the deterministic checks included in the repository. The test plan combines this baseline with requirements-based functional, integration, usability and acceptance coverage to support a structured evaluation of the MVP.